Dictionary (Detail Explanation)

📘 Topic 1: Basics of Python Dictionary

◆ 1.1 What is a Dictionary?

- A **dictionary** in Python is a **collection of key-value pairs**.
- Each **key** is unique and maps to a **value**.
- It is one of Python's **built-in data types**, used for storing data like a real-life dictionary (word → meaning).

📌 Example (real-world analogy):

```
dictionary = {
    "apple": "a fruit",
    "car": "a vehicle",
    "pen": "a writing tool"
}
```

Here:

- Keys → `"apple"`, `"car"`, `"pen"`
- Values → `"a fruit"`, `"a vehicle"`, `"a writing tool"`

◆ 1.2 Properties of Dictionary

- **Unordered (before Python 3.7)**: No guaranteed order.
- **Ordered (Python 3.7 and later)**: Maintains **insertion order**.
- **Mutable**: You can change, add, or remove items.
- **Indexed by keys, not positions**: Unlike lists/tuples, which use numeric indexes.

📌 Example:

```
person = {"name": "Nusrat", "age": 28}
print(person["name"])   # Access by key, not index → Nusrat
```

◆ 1.3 Creating a Dictionary

There are many ways to create a dictionary:

1. **Using curly braces `{}`**
2. `my_dict = {"name": "Alice", "age": 25}`
3. **Using `dict()` constructor**
4. `my_dict = dict(name="Alice", age=25)`
5. **Empty dictionary**
6. `empty = {}`
7. `empty2 = dict()`
8. **From a list of tuples**
9. `my_dict = dict([("name", "Alice"), ("age", 25)])`
10. **From two lists (zip)**
11. `keys = ["a", "b", "c"]`
12. `values = [1, 2, 3]`
13. `my_dict = dict(zip(keys, values))`

◆ 1.4 Rules for Keys and Values

- **Keys**
  - Must be **unique** (duplicate keys overwrite old value).
  - Must be **immutable** (string, number, tuple).
  - ❌ Lists and dicts cannot be keys.
- **Values**
  - Can be of **any type** (mutable or immutable).
  - Duplicates allowed.

📌 Example:

```
d = {
    "a": 1,
    "b": [1,2,3],        # list value allowed
    (1,2): "tuple key"   # tuple key allowed
}
```

◆ 1.5 Dictionary vs Other Data Structures

Feature	List	Tuple	Set	Dictionary
Ordered	✅	✅	❌	✅ (Python ≥ 3.7)
Mutable	✅	❌	✅	✅
Indexing	Numeric	Numeric	❌	Key-based
Duplicates	✅	✅	❌	Keys ❌, Values ✅

✅ **Summary of Topic 1**:

- Dictionary = collection of **key-value pairs**.

- **Mutable**, **ordered** (Python ≥ 3.7).
- Keys must be **unique & immutable**, values can be anything.
- Created using `{}` or `dict()`.

⍰ Topic 2: Dictionary Keys and Values

◆ 2.1 Keys in Dictionary

Rules for Keys

1. **Unique** – no duplicate keys (if used, last one overwrites).
2. **Immutable** – must be of immutable type:
   - Allowed: `str`, `int`, `float`, `bool`, `tuple`
   - Not allowed: `list`, `dict`, `set`
3. **Hashable** – Python uses a hash table internally, so keys must be hashable.

📌 Example:

```
d = {"a": 1, "b": 2, "a": 3}
print(d)   # {'a': 3, 'b': 2} → last "a" overwrote previous
```

📌 Invalid key:

```
d = {[1,2,3]: "list"}  # ✕ Error: list is unhashable
```

📌 Valid tuple key:

```
d = {(1,2): "point", (3,4): "coordinates"}
print(d[(1,2)])  # point
```

◆ 2.2 Values in Dictionary

Rules for Values

1. Values can be **any data type**.
2. Values can be **mutable or immutable**.
3. Duplicate values are **allowed**.

📌 Examples:

```
d = {
    "x": 100,           # int
    "y": [1, 2, 3],     # list (mutable)
    "z": {"a": 1},      # dict inside dict
    "p": (5, 6)         # tuple
}
```

📌 Duplicate values:

```
d = {"a": 1, "b": 1, "c": 1}
print(d)  # {'a': 1, 'b': 1, 'c': 1} ✅ Allowed
```

◆ 2.3 Accessing Keys & Values

By Key

```
person = {"name": "Nusrat", "age": 28}
print(person["name"])   # Nusrat
```

⚠ If key doesn't exist → `KeyError`:

```
print(person["salary"])   # ❌ Error
```

Using `.get()`

```
print(person.get("name"))       # Nusrat
print(person.get("salary"))     # None (no error)
print(person.get("salary", 0))  # 0 (default value)
```

◆ 2.4 Extracting Keys and Values

- **Keys only**
- `d = {"a":1, "b":2, "c":3}`
- `print(d.keys())  # dict_keys(['a', 'b', 'c'])`
- **Values only**
- `print(d.values())  # dict_values([1, 2, 3])`
- **Both (items)**
- `print(d.items())   # dict_items([('a', 1), ('b', 2), ('c', 3)])`

⚡ Note: `keys()`, `values()`, `items()` return **views** (dynamic, update when dict changes).

📌 Example:

```
d = {"x": 1}
k = d.keys()
print(k)   # dict_keys(['x'])
d["y"] = 2
print(k)   # dict_keys(['x','y']) → updated automatically
```

◆ 2.5 Checking Existence

- Check if key exists:
- `d = {"a":1, "b":2}`
- `print("a" in d)  # True`
- `print("c" in d)  # False`
- Check if value exists:
- `print(1 in d.values())  # True`

◆ 2.6 Updating Keys & Values

- Update value:
- `d = {"name": "Nusrat", "age": 28}`
- `d["age"] = 29`
- Add new key-value:
- `d["city"] = "Dhaka"`

◆ 2.7 Summary (Keys vs Values)

Feature	Keys	Values
Duplicates	❌ Not allowed	✅ Allowed
Type	Must be immutable & hashable	Any type (mutable/immutable)
Example	`"a", 1, (1,2)`	`"hello", [1,2,3], {"x":1}`
Access	`dict[key]`	`dict.values()`

✅ **Summary of Topic 2:**

- Keys: **unique, immutable, hashable**.
- Values: **any type, duplicates allowed**.
- Access via `dict[key]` or `get()`.
- Use `keys()`, `values()`, `items()` for extraction.
- Keys are like **indexes**, values are the **stored data**.

⍰ Topic 3: Accessing Elements in Dictionary

◆ 3.1 Accessing by Key (Direct Indexing)

- Syntax:
- `dict_name[key]`
- Example:
- `person = {"name": "Nusrat", "age": 28}`
- `print(person["name"])  # Nusrat`
- `print(person["age"])  # 28`

⚠ If the key is **not present**, Python raises a `KeyError`.

```
print(person["salary"])  # ✕ KeyError
```

◆ 3.2 Using `.get()` (Safe Access)

- Syntax:
- `dict_name.get(key, default_value)`
- Example:
- `person = {"name": "Nusrat", "age": 28}`
- `print(person.get("name"))  # Nusrat`
- `print(person.get("salary"))  # None (no error)`
- `print(person.get("salary", 0))  # 0 (default)`

✅ Advantage: Avoids `KeyError`.

◆ 3.3 Accessing All Keys

- `keys()` → returns all keys (as a **view object**).
- `person = {"name": "Nusrat", "age": 28}`
- `print(person.keys())  # dict_keys(['name', 'age'])`
- Can loop over it:
- `for key in person.keys():`
- `    print(key)`

◆ 3.4 Accessing All Values

- `values()` → returns all values.
- `print(person.values())  # dict_values(['Nusrat', 28])`
- Looping:
- `for value in person.values():`
- `    print(value)`

◆ 3.5 Accessing Key-Value Pairs

- `items()` → returns key-value pairs as tuples.
- `print(person.items())`
- `# dict_items([('name', 'Nusrat'), ('age', 28)])`
- Looping:
- `for k, v in person.items():`
- `    print(k, v)`

◆ 3.6 Access with `in` Operator

- Check if a **key exists**:
- `print("name" in person)    # True`
- `print("salary" in person)  # False`
- Check if a **value exists**:
- `print(28 in person.values())  # True`

◆ 3.7 Accessing Nested Dictionaries

If dictionary contains another dictionary:

```
students = {
    "s1": {"name": "Nusrat", "age": 28},
    "s2": {"name": "Korayil", "age": 30}
}

print(students["s1"]["name"])   # Nusrat
```

◆ 3.8 Access with `dict.get()` + Nested

- Safe way with nested dictionary:

```
print(students.get("s1", {}).get("age", "Not Found"))
# 28
print(students.get("s3", {}).get("age", "Not Found"))
# Not Found
```

◆ 3.9 Dictionary Views are Dynamic

When you use `.keys()`, `.values()`, `.items()`, they reflect changes immediately.

📌 Example:

```
d = {"a": 1}
k = d.keys()
print(k)   # dict_keys(['a'])
d["b"] = 2
print(k)   # dict_keys(['a', 'b']) → auto updated
```

◆ 3.10 Summary

- `dict[key]` → Direct access, raises `KeyError` if missing.
- `.get(key, default)` → Safe access (returns `None` or default).
- `.keys()` → All keys (dynamic view).
- `.values()` → All values (dynamic view).
- `.items()` → Key-value pairs (dynamic view).
- `in` operator → Check existence.
- Works even with **nested dictionaries**.

⍰ Topic 4: Adding & Updating Dictionary Elements

◆ 4.1 Adding New Elements

You can add a new key-value pair simply by assigning to a new key.

📌 Example:

```
person = {"name": "Nusrat", "age": 28}
person["city"] = "Dhaka"   # Adds new key
print(person)
# {'name': 'Nusrat', 'age': 28, 'city': 'Dhaka'}
```

◆ 4.2 Updating Existing Elements

If the key already exists, assigning a new value will **update** it.

📌 Example:

```
person["age"] = 29   # Updates 'age'
print(person)
# {'name': 'Nusrat', 'age': 29, 'city': 'Dhaka'}
```

◆ 4.3 Using `.update()` Method

- `update()` can add multiple key-value pairs at once.
- It can also overwrite existing keys.

📌 Example 1 (add new keys):

```
person.update({"country": "Bangladesh", "job": "Engineer"})
print(person)
# {'name': 'Nusrat', 'age': 29, 'city': 'Dhaka', 'country': 'Bangladesh',
'job': 'Engineer'}
```

📌 Example 2 (update existing key):

```
person.update({"age": 30})
print(person)
# {'name': 'Nusrat', 'age': 30, 'city': 'Dhaka', 'country': 'Bangladesh',
'job': 'Engineer'}
```

◆ 4.4 Adding with `setdefault()`

- Adds a key with a default value **only if it doesn't exist**.
- If the key exists, it **does nothing**.

📌 Example:

```
d = {"a": 1, "b": 2}
d.setdefault("c", 3)
print(d)  # {'a': 1, 'b': 2, 'c': 3}

d.setdefault("a", 100)
print(d)  # {'a': 1, 'b': 2, 'c': 3} → did not update existing key
```

◆ 4.5 Adding/Updating Nested Dictionary Elements

When dictionary has another dictionary inside it:

📌 Example:

```
students = {
    "s1": {"name": "Nusrat", "age": 28},
    "s2": {"name": "Korayil", "age": 30}
}

# Add new key-value to s1
students["s1"]["city"] = "Dhaka"

# Update age
students["s2"]["age"] = 31

print(students)
```

Output:

```
{
  's1': {'name': 'Nusrat', 'age': 28, 'city': 'Dhaka'},
  's2': {'name': 'Korayil', 'age': 31}
}
```

◆ 4.6 Adding/Updating with Dictionary Merging (Python 3.9+)

You can use the **merge operator** `|` or **update operator** `|=`.

📌 Example:

```
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}

merged = d1 | d2
print(merged)  # {'a': 1, 'b': 3, 'c': 4}

d1 |= d2
print(d1)      # {'a': 1, 'b': 3, 'c': 4}
```

◆ 4.7 Summary

- `dict[key] = value` → Add or update single element.

- `.update({...})` → Add/update multiple elements.
- `.setdefault(key, default)` → Add key only if missing.
- `dict1 | dict2` (Python 3.9+) → Merge dictionaries.
- Works the same for **nested dictionaries**, but you must specify the sub-dictionary.

□ Topic 5: Removing Elements in Dictionary

◆ 5.1 Using `pop(key)`

- Removes the element with the specified key.
- Returns the value of the removed key.
- If the key does not exist → raises `KeyError`.

📌 Example:

```
person = {"name": "Nusrat", "age": 28, "city": "Dhaka"}
removed_value = person.pop("city")
print(removed_value)   # Dhaka
print(person)          # {'name': 'Nusrat', 'age': 28}
```

📌 With default value (avoids error):

```
person.pop("job", "Not Found")  # "Not Found"
```

◆ 5.2 Using `popitem()`

- Removes and returns the **last inserted (key, value) pair** as a tuple.
- From Python 3.7+, dictionaries maintain insertion order.

📌 Example:

```
person = {"name": "Nusrat", "age": 28, "city": "Dhaka"}
print(person.popitem())   # ('city', 'Dhaka')
print(person)             # {'name': 'Nusrat', 'age': 28}
```

◆ 5.3 Using `del` Keyword

- Deletes a key-value pair.

- Can also delete the entire dictionary.

📌 Example:

```
person = {"name": "Nusrat", "age": 28}
del person["age"]
print(person)   # {'name': 'Nusrat'}
```

📌 Delete whole dictionary:

```
del person
# Now "person" no longer exists
```

◆ 5.4 Using `clear()`

- Removes **all elements**, leaving an empty dictionary.

📌 Example:

```
person = {"name": "Nusrat", "age": 28}
person.clear()
print(person)   # {}
```

◆ 5.5 Safe Removal (Checking Before Delete)

📌 Example:

```
person = {"name": "Nusrat", "age": 28}
if "city" in person:
    del person["city"]
else:
    print("Key not found")
```

◆ 5.6 Removing from Nested Dictionary

📌 Example:

```
students = {
    "s1": {"name": "Nusrat", "age": 28, "city": "Dhaka"},
    "s2": {"name": "Korayil", "age": 30}
}

# Remove 'city' from s1
students["s1"].pop("city")
```

```
print(students)
```

◆ 5.7 Summary

Method	Removes…	Returns	Error if key missing?
`pop(key)`	Specified key	Value	✅ Yes (unless default)
`popitem()`	Last inserted key-value pair	(key, value)	✅ Yes
`del dict[key]`	Specified key	Nothing	✅ Yes
`clear()`	All key-value pairs (empties dict)	Nothing	❌ No
`del dict`	Entire dictionary object	N/A	✅ Yes (if accessed later)

◆ Topic 06: Dictionary Methods – Full Overview

Python dictionaries come with **built-in methods** to manipulate, access, and manage key-value pairs efficiently.

Here's a complete breakdown:

---

1. Accessing Methods

Method	Description	Example
`dict.get(key, default)`	Returns value for key if exists, else default (None if not given)	`d.get("a", 0)`
`dict.keys()`	Returns a view of all keys	`d.keys()` → `dict_keys([...])`
`dict.values()`	Returns a view of all values	`d.values()` → `dict_values([...])`
`dict.items()`	Returns view of key-value pairs	`d.items()` → `dict_items([...])`

---

2. Adding & Updating Methods

Method	Description	Example
`dict[key] = value`	Add new key or update existing key	`d["x"] = 10`
`dict.update(other_dict)`	Update with another dict (adds new keys or updates existing ones)	`d.update({"y": 20})`
`dict.setdefault(key, default)`	Add key with default value if key does not exist	`d.setdefault("z", 30)`

---

3. Removing Methods

Method	Description	Example
`dict.pop(key)`	Remove key and return value	`d.pop("x")`
`dict.pop(key, default)`	Remove key if exists else return default	`d.pop("k", 0)`
`dict.popitem()`	Remove last inserted key-value pair	`d.popitem()`
`del dict[key]`	Remove key-value pair	`del d["x"]`
`dict.clear()`	Remove all items	`d.clear()`

---

4. Copying Methods

Method	Description	Example
`dict.copy()`	Returns a shallow copy of dictionary	`new_dict = d.copy()`
`copy.deepcopy(dict)`	Deep copy (import `copy` module)	`import copy; new_dict = copy.deepcopy(d)`

---

5. Other Useful Methods

Method	Description	Example
`dict.fromkeys(seq, value)`	Create dictionary with keys from sequence, same value	`dict.fromkeys(["a","b"],0)` → `{'a':0,'b':0}`
`len(dict)`	Number of key-value pairs	`len(d)`
`in` operator	Check if key exists	`"a" in d` → True

`not in` operator	Check if key does not exist	`"z" not in d` → True

◆ Topic 09: Nested Dictionaries

A **nested dictionary** is a dictionary where the **values themselves can be dictionaries**.
This is useful for representing complex structured data.

---

1. Creating a Nested Dictionary

```
students = {
    "s1": {"name": "Nusrat", "age": 28, "course": "CS"},
    "s2": {"name": "Korayil", "age": 30, "course": "AI"}
}

print(students)
```

Output:

```
{
  's1': {'name': 'Nusrat', 'age': 28, 'course': 'CS'},
  's2': {'name': 'Korayil', 'age': 30, 'course': 'AI'}
}
```

---

2. Accessing Elements in Nested Dictionary

```
print(students["s1"]["name"])   # Nusrat
print(students["s2"]["course"]) # AI
```

✅ Use `.get()` for safety:

```
print(students.get("s3", {}).get("name", "Not Found"))  # Not Found
```

---

3. Adding a New Nested Key-Value

```
students["s1"]["city"] = "Dhaka"
students["s3"] = {"name": "Ali", "age": 25, "course": "ML"}

print(students)
```

---

4. Updating Nested Values

```
students["s2"]["age"] = 31
print(students["s2"])
# {'name': 'Korayil', 'age': 31, 'course': 'AI'}
```

---

5. Removing Nested Elements

```
del students["s1"]["course"]      # Remove inner key
students["s2"].pop("age")         # Remove using pop
print(students)
```

---

6. Iterating Nested Dictionaries

```
for student_id, info in students.items():
    print(f"ID: {student_id}")
    for key, value in info.items():
        print(f"  {key} : {value}")
```

Output Example:

```
ID: s1
  name : Nusrat
  age : 28
  city : Dhaka
ID: s2
  name : Korayil
  course : AI
```

---

7. Summary

- Nested dictionary = dictionary **inside another dictionary**.
- Access nested items: `dict[key1][key2]`.
- Add/update/remove works the same, just specify inner dictionary.
- Iteration can be **two-level loops** for outer & inner dictionaries.

◆ Topic 11: Dictionary Advanced Features

1) **Dictionary Comprehension: এক লাইনে dictionary বানানো যায় (list comprehension–এর মতো)।**
2) **Dictionary Merging (Python 3.9+)**

দুইটা dictionary merge করা যায় খুব সহজে

3) Dictionary Unpacking (**)

একটা dictionary–র key–value অন্য dictionary–তে merge করতে:

```
base = {"name": "Nusrat"}
extra = {"age": 28, "city": "Dhaka"}

profile = {**base, **extra}
print(profile)
# {'name': 'Nusrat', 'age': 28, 'city': 'Dhaka'}
```

4) `fromkeys()` Method

একই value সহ dictionary বানানো যায়:

```
keys = ["a", "b", "c"]
d = dict.fromkeys(keys, 0)
print(d)
# {'a': 0, 'b': 0, 'c': 0}
```

5) `setdefault()` Method

Key না থাকলে নতুন key যোগ করবে, থাকলে কিছু করবে না।

```
d = {"x": 10}
d.setdefault("y", 20)
print(d)
# {'x': 10, 'y': 20}
```

6) `defaultdict` (from `collections`)

Normal dict–এ key না থাকলে error দেয়।
কিন্তু `defaultdict` missing key–র জন্য default value বানায়।

```
from collections import defaultdict

d = defaultdict(int)   # missing key → 0 automatically
d["a"]
print(d)# defaultdict(<class 'int'>, {'a': 0})
d["a"] += 1
print(d)  # {'a': 1}
```

☞ Data counting এ খুব কাজে লাগে।

7) `Counter` (from `collections`)

Dictionary–র উপর ভিত্তি করে **frequency counter** বানায়।

```
from collections import Counter

text = "banana"
count = Counter(text)
print(count)
# {'a': 3, 'b': 1, 'n': 2}
```